

Résumé Java Scripte

1 Définition de JavaScript

JavaScript (JS) est un langage de programmation utilisé principalement pour rendre les pages web interactives (animations, formulaires dynamiques, etc.).

2 Utilité de JavaScript

- Manipulation des éléments HTML et CSS
- Création d'interactions dynamiques (animations, validation de formulaires, etc.)
- Communication avec un serveur via AJAX ou Fetch API
- Développement d'applications web modernes avec des frameworks comme React, Vue ou Angular

3. Comment utiliser JavaScript ?

On peut intégrer JavaScript dans une page web de trois manières :

- **Directement dans une balise <script>**

```
<script>
  alert("Bienvenue en JavaScript !");
</script>
```

- **Dans un fichier externe .js :**

```
<script src="script.js"></script>
```

- **Dans une console de navigateur (F12 → Console)**

4 Affichage dans la console

Permet d'afficher des messages ou des valeurs dans la console du navigateur.

```
console.log("Bonjour, ceci est un message dans la console.");
```

5 Affiche du texte dans la page web.

Permet d'écrire directement dans le document HTML.

```
document.write("<h1>Bonjour, JavaScript !</h1>");
```

6 l'instruction Alert

L'instruction `alert()` est utilisée pour afficher une boîte de dialogue avec un message à l'utilisateur. Elle interrompt l'exécution du script jusqu'à ce que l'utilisateur clique sur "OK".

```
alert("Message à afficher");
```

7 l'instruction Prompt

Demande à l'utilisateur de saisir une valeur.

```
let nom = prompt("Quel est votre nom ?");  
console.log("Bonjour " + nom);
```

8 Commentaires en JavaScript

- **Commentaire sur une ligne :**

```
// Ceci est un commentaire
```

- **Commentaire sur plusieurs lignes :**

```
/*  
Ceci est un commentaire  
sur plusieurs lignes  
*/
```

9 Variables (var, let) et différences

- **var** (ancienne façon de déclarer une variable, portée globale)

```
var x = 10;
```

- **let** (recommandé, portée limitée au bloc où elle est déclarée)

```
let y = 20;
```

10 Opérateurs arithmétiques

E

Opérateur	Description	Exemple
+	Addition	5 + 2 → 7
-	Soustraction	5 - 2 → 3
*	Multiplication	5 * 2 → 10
/	Division	5 / 2 → 2.5
%	Modulo (reste)	5 % 2 → 1
**	Puissance	5 ** 2 → 25

11. Les opérateurs logiques

OPERATEUR	DESCRIPTION	EXEMPLE
&&	ET Logique	(true && false) → false
	Ou Logique	(true false) → true
!	Non Logique	!true → false

12 Opérateurs de comparaison

Opérateur	Description	Exemple
==	Égalité (vérifie seulement la valeur)	"5" == 5 → true
===	Égalité stricte (vérifie valeur et type)	"5" === 5 → false
!=	Différent	5 != 3 → true
!==	Différent strict	"5" !== 5 → true
<	Inférieur	5 < 10 → true
>	Supérieur	5 > 2 → true
<=	Inférieur ou égal	5 <= 5 → true
>=	Supérieur ou égal	5 >= 3 → true

13 La conversion en JavaScript

- **Avec Number()**

Convertit une chaîne en nombre entier ou décimal.

```
let num = Number("42");  
console.log(num); // 42 (nombre)
```

- Avec `parseInt()`

Convertit une chaîne en nombre entier uniquement.

```
let entier = parseInt("42.99");
console.log(entier); // 42
```

- Avec `parseFloat()`

Convertit une chaîne en nombre décimal.

```
let decimal = parseFloat("42.99");
console.log(decimal); // 42.99
```

Conversion en chaîne

- Avec `String()`

```
let num = 100;
let str = String(num);
console.log(str); // "100"
console.log(typeof str); // "string"
```

- Avec `.toString()`

```
let num2 = 50;
console.log(num2.toString()); // "50"
```

14 Les structures de contrôle

- if simple

```
let age = 18;
if (age >= 18) {
  console.log("Vous êtes majeur.");
}
```

- if else

```
let heure = 10;
if (heure < 12) {
  console.log("Bonjour !");
} else {
  console.log("Bonsoir !");
}
```

- **if - else if – else**

```
let note = 85;

if (note >= 90) {
  console.log("Excellent !");
} else if (note >= 75) {
  console.log("Très bien !");
} else if (note >= 50) {
  console.log("Passable.");
} else {
  console.log("Échec !");
}
```

- **L'opérateur ternaire :**

L'opérateur ternaire est une façon concise d'écrire une condition if-else. Il utilise le symbole ? et a la structure suivante :

javascript

```
condition ? valeur_si_vrai : valeur_si_faux;
```

- **condition** : Une expression qui retourne **true** ou **false**.
- **valeur_si_vrai** : Valeur retournée si la condition est vraie.
- **valeur_si_faux** : Valeur retournée si la condition est fausse.

Les boucles

En JavaScript, les **boucles** permettent d'exécuter un bloc de code plusieurs fois tant qu'une condition est remplie. Il existe plusieurs types de boucles :

1. La boucle for

La boucle for est utilisée lorsqu'on connaît à l'avance le nombre d'itérations.

Syntaxe :

```
javascript

for (initialisation; condition; incrémentation) {
    // Code à exécuter
}
```

Exemple :

```
for (let i = 0; i < 5; i++) {
    console.log("Itération n°", i);
}
```

💡 Explication :

- `let i = 0` → Initialise `i` à 0.
- `i < 5` → Exécute le bloc tant que `i` est inférieur à 5.
- `i++` → Incrémente `i` après chaque tour.

2. La boucle while

La boucle while s'exécute tant que la condition est vraie.

Syntaxe :

```
javascript

while (condition) {
    // Code à exécuter
}
```

Exemple :

```
let compteur = 0;
while (compteur < 3) {
    console.log("Compteur =", compteur);
    compteur++; // Incrémentation
}
```

💡 Explication :

- La boucle tourne tant que `compteur < 3`.
- À chaque itération, `compteur` est incrémenté.

3 . La boucle do ... while

Cette boucle fonctionne comme while, mais **elle s'exécute au moins une fois**, même si la condition est fausse.

Syntaxe :

```
do {  
    // Code à exécuter  
} while (condition);
```

Exemple :

```
let num = 5;  
do {  
    console.log("Num =", num);  
    num--;  
} while (num > 3);
```

💡 **Explication :**

- La boucle s'exécute une première fois même si `num > 3` est faux.

4.La boucle For ... in

Utilisée pour parcourir les propriétés d'un objet.

Syntaxe :

```
for (let clé in objet) {  
    // Bloc de code à exécuter  
}
```

Exemple :

```
let personne = {nom: "Ayoub", âge: 25, ville: "Casablanca"};  
for (let clé in personne) {  
    console.log(clé + " : " + personne[clé]);  
}
```

5. La boucle for...of

Utilisée pour parcourir les éléments d'un tableau ou d'une chaîne de caractères.

Syntaxe :

```
for (let valeur of iterable) {  
    // Bloc de code à exécuter  
}
```

Exemple :

```
let fruits = ["Pomme", "Banane", "Orange"];  
for (let fruit of fruits) {  
    console.log(fruit);  
}
```

6. La boucle forEach

La méthode `forEach` est utilisée pour parcourir les éléments d'un tableau et exécuter une fonction de rappel (callback) sur chaque élément.

Syntaxe :

```
tableau.forEach(function(élément, index, tableau) {  
    // Bloc de code à exécuter  
});
```

- `élément` : La valeur actuelle de l'élément du tableau.
- `index` (*optionnel*) : L'index de l'élément en cours de traitement.
- `tableau` (*optionnel*) : Le tableau sur lequel `forEach` est appliqué.

💡 La méthode `forEach` ne retourne pas de valeur et ne permet pas d'interrompre la boucle avec `break` ou `return`.

Exemple 1 : Parcourir un tableau


```
let nombres = [10, 20, 30, 40];

nombres.forEach(function(nombre) {
    console.log("Valeur :", nombre);
});
```

◆ Résultat :

```
yaml

Valeur : 10
Valeur : 20
Valeur : 30
Valeur : 40
```

Exemple 2 : Accéder à l'index des éléments

```
let fruits = ["Pomme", "Banane", "Orange"];

fruits.forEach(function(fruit, index) {
    console.log("Index :", index, "- Fruit :", fruit);
});
```

◆ Résultat :

```
yaml

Index : 0 - Fruit : Pomme
Index : 1 - Fruit : Banane
Index : 2 - Fruit : Orange
```

Les Tableaux

Un **tableau** en JavaScript est une structure de données qui permet de stocker plusieurs valeurs dans une seule variable. Il peut contenir des types de données variés : nombres, chaînes de caractères, objets, etc.

1. Déclaration d'un Tableau

Avec `[]` (notation littérale) :

```
javascript  
  
let fruits = ["Pomme", "Banane", "Orange"];
```

2. Accéder aux éléments d'un tableau

Chaque élément du tableau a un **indice** (index), qui commence à `0`.

```
javascript  
  
console.log(fruits[0]); // "Pomme"  
console.log(fruits[2]); // "Orange"
```

💡 **Attention** : Un index hors limite retourne `undefined` :

```
javascript  
  
console.log(fruits[5]); // undefined
```

3. Modifier un élément d'un tableau

```
fruits[1] = "Mangue";  
console.log(fruits); // ["Pomme", "Mangue", "Orange"]
```

4. Boucles sur les Tableaux

```
javascript

for (let i = 0; i < fruits.length; i++) {
    console.log(fruits[i]);
}
```

5. Fonctions et Méthodes Utilisées avec les Tableaux

- **push()** – Ajouter un élément à la fin du tableau

Cette méthode ajoute un ou plusieurs éléments à la fin d'un tableau et retourne la nouvelle longueur du tableau.

- ♦ **Syntaxe :**

```
javascript

tableau.push(element1, element2, ...);
```

- ♦ **Exemple :**

```
javascript

let fruits = ["Pomme", "Banane"];
fruits.push("Orange");
console.log(fruits); // ["Pomme", "Banane", "Orange"]
```

- **pop()** – Supprimer le dernier élément du tableau

Cette méthode supprime le dernier élément d'un tableau et retourne cet élément supprimé

◆ **Syntaxe :**

```
javascript  
  
tableau.pop();
```

◆ **Exemple :**

```
javascript  
  
let fruits = ["Pomme", "Banane", "Orange"];  
let dernierFruit = fruits.pop();  
console.log(fruits); // ["Pomme", "Banane"]  
console.log(dernierFruit); // "Orange"
```

- **unshift() – Ajouter un élément au début du tableau**

Cette méthode ajoute un ou plusieurs éléments au début du tableau et retourne la nouvelle longueur du tableau.

◆ **Syntaxe :**

```
javascript  
  
tableau.unshift(element1, element2, ...);
```

◆ **Exemple :**

```
javascript  
  
let fruits = ["Banane", "Orange"];  
fruits.unshift("Pomme");  
console.log(fruits); // ["Pomme", "Banane", "Orange"]
```

- **shift() – Supprimer le premier élément du tableau**

Cette méthode supprime le premier élément du tableau et retourne cet élément supprimé.

◆ **Syntaxe :**

```
javascript

tableau.shift();
```

◆ **Exemple :**

```
javascript

let fruits = ["Pomme", "Banane", "Orange"];
let premierFruit = fruits.shift();
console.log(fruits); // ["Banane", "Orange"]
console.log(premierFruit); // "Pomme"
```

- **indexOf() – Trouver l'index d'un élément**

Cette méthode retourne l'index du premier élément trouvé dans le tableau.

Si l'élément n'existe pas, elle retourne -1.

◆ **Syntaxe :**

```
javascript

tableau.indexOf(element);
```

◆ **Exemple :**

```
javascript

let fruits = ["Pomme", "Banane", "Orange"];
console.log(fruits.indexOf("Banane")); // 1
console.log(fruits.indexOf("Kiwi")); // -1 (car "Kiwi" n'existe pas)
```

- **includes()** – Vérifier si un élément existe dans le tableau

Cette méthode retourne true si l'élément existe dans le tableau, sinon false.

- ◆ Syntaxe :

```
javascript

tableau.includes(element);
```

- ◆ Exemple :

```
javascript

let fruits = ["Pomme", "Banane", "Orange"];
console.log(fruits.includes("Banane")); // true
console.log(fruits.includes("Kiwi"));  // false
```

- **concat()** – Fusionner plusieurs tableaux

Cette méthode permet de combiner plusieurs tableaux en un seul sans modifier les originaux.

```
let nouveauTableau = tableau1.concat(tableau2, tableau3, ...);
```

- ◆ Exemple :

```
javascript

let fruits = ["Pomme", "Banane"];
let autresFruits = ["Orange", "Kiwi"];
let tousLesFruits = fruits.concat(autresFruits);
console.log(tousLesFruits); // ["Pomme", "Banane", "Orange", "Kiwi"]
```

- **length** – Obtenir la longueur du tableau

Cette propriété retourne le nombre d'éléments présents dans le tableau.

◆ **Syntaxe :**

```
javascript  
  
tableau.length;
```

◆ **Exemple :**

```
javascript  
  
let fruits = ["Pomme", "Banane", "Orange"];  
console.log(fruits.length); // 3
```

Méthode / Propriété	Description	Exemple
<code>push()</code>	Ajoute un élément à la fin du tableau	<code>fruits.push("Kiwi");</code>
<code>pop()</code>	Supprime le dernier élément	<code>fruits.pop();</code>
<code>unshift()</code>	Ajoute un élément au début du tableau	<code>fruits.unshift("Mangue");</code>
<code>shift()</code>	Supprime le premier élément	<code>fruits.shift();</code>
<code>indexOf()</code>	Trouve l'index d'un élément	<code>fruits.indexOf("Banane");</code>
<code>includes()</code>	Vérifie si un élément existe	<code>fruits.includes("Orange");</code>
<code>concat()</code>	Fusionne plusieurs tableaux	<code>fruits.concat(autresFruits);</code>
<code>length</code>	Retourne la taille du tableau	<code>fruits.length;</code>

Les Fonction

Une fonction est un bloc de code réutilisable qui effectue une tâche spécifique. Elle peut prendre des entrées (paramètres), exécuter des opérations et retourner un résultat.

Au lieu de répéter le même code plusieurs fois, une fonction permet de l'écrire une seule fois et de l'appeler autant de fois que nécessaire.

Les fonctions sans paramètres :

Une fonction sans paramètres ne prend aucune valeur en entrée lorsqu'elle est appelée. Elle exécute son code sans avoir besoin d'arguments spécifiques.

Syntaxe

```
javascript

function nomDeFonction() {
    // Code à exécuter
}
```

Exemple

```
javascript

function saluer() {
    console.log("Bonjour !");
}

saluer(); // Affiche "Bonjour !"
```

Les fonctions avec paramètres

Une fonction avec paramètres accepte des valeurs (arguments) lors de son appel. Ces valeurs sont utilisées à l'intérieur de la fonction pour effectuer des opérations.

Syntaxe :

```
function nomDeFonction(param1, param2) {
    // Code qui utilise Les paramètres
}
```


Exemple :

```
function additionner(a, b) {  
    console.log("Résultat :", a + b);  
}  
  
additionner(3, 5); // Affiche "Résultat : 8"
```

Passage par valeur

Le passage par valeur concerne les types primitifs (nombres, chaînes de caractères, booléens, etc.). Lorsque vous passez une variable primitive à une fonction, une copie de sa valeur est créée. Les modifications dans la fonction n'affectent pas la valeur originale.

```
function doubler(x) {  
    x = x * 2;  
    console.log("Valeur dans la fonction :", x);  
}  
  
let nombre = 10;  
doubler(nombre);  
console.log("Valeur originale :", nombre); // La valeur de 'nombre' reste 10
```

Passage par référence

Le passage par référence concerne les objets et tableaux. Lorsqu'un objet ou un tableau est passé à une fonction, une référence à cet objet est envoyée. Cela signifie que toute modification de cet objet ou tableau affectera directement l'original.

```
function ajouterElement(tab) {  
    tab.push(100);  
    console.log("Tableau dans la fonction :", tab);  
}  
  
let tableau = [1, 2, 3];  
ajouterElement(tableau);  
console.log("Tableau original :", tableau); // Le tableau est modifié
```

Les fonctions avec return

Une fonction avec return permet de renvoyer un résultat à l'appelant. Elle permet d'effectuer un calcul ou une opération et de retourner la valeur au code appelant.

Syntaxe :

```
function nomDeFonction(param1, param2) {  
    return valeur; // Retourne un résultat  
}
```

Exemple :

```
function multiplier(a, b) {  
    return a * b;  
}  
  
let resultat = multiplier(4, 5);  
console.log("Résultat :", resultat); // Affiche "Résultat : 20"
```

Les fonctions fléchées (Arrow Functions)

Les fonctions fléchées sont une syntaxe plus concise pour écrire des fonctions.

Version complète (avec accolades et return) :

```
const additionner = (a, b) => {  
    return a + b;  
};  
  
console.log(additionner(5, 3)); // Affiche 8
```

Version simplifiée (une seule ligne) :

```
const additionner = (a, b) => a + b;  
  
console.log(additionner(5, 3)); // Affiche 8
```

la méthode filter() en JavaScript

La méthode filter() est utilisée pour créer un **nouveau tableau** contenant **uniquement les éléments qui remplissent une condition spécifique**.

Syntaxe :

```
let nouveauTableau = tableauOriginal.filter(function(element, index, array) {  
    // Condition à vérifier  
    return condition;  
});
```

Paramètres :

- element : Élément en cours de traitement.
- index (*optionnel*) : Index de l'élément dans le tableau.
- array (*optionnel*) : Tableau d'origine.
- condition : Une expression qui retourne true (garde l'élément) ou false (exclut l'élément).

⚡ **Retourne :** Un **nouveau tableau** contenant les éléments qui remplissent la condition.

Exemple 1 : Filtrer les nombres supérieurs à 50 :

```
let nombres = [10, 55, 80, 45, 100];

let nombresFiltres = nombres.filter(nombre => nombre > 50);

console.log(nombresFiltres);
```

Résultat : [55, 80, 100]

Seuls les nombres supérieurs à 50 sont conservés.

la Fonction map en JavaScript

La méthode .map() est une **fonction des tableaux** en JavaScript. Elle permet de **transformer chaque élément** d'un tableau **en un nouveau tableau**, sans modifier le tableau d'origine.

Syntaxe :

```
tableau.map(function(element, index, array) {  
    // Retourner une nouvelle valeur  
});
```

Exemple 1:

```
const nombres = [1, 2, 3, 4];

const doubles = nombres.map((nombre) => {  
    return nombre * 2;  
});

console.log(doubles); // [2, 4, 6, 8]
```

Exemple 2 :

```
const utilisateurs = [  
    { nom: "Ali", age: 25 },  
    { nom: "Sara", age: 30 }  
];

const noms = utilisateurs.map((utilisateur) => utilisateur.nom);

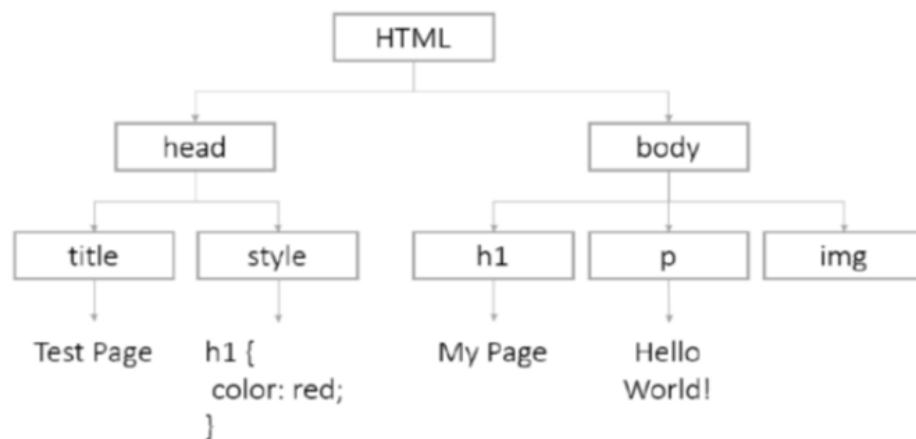
console.log(noms); // ["Ali", "Sara"]
```

Définition du DOM en JavaScript

Le DOM (Document Object Model) est une interface qui permet d'accéder et de manipuler le contenu HTML d'une page web à l'aide de JavaScript. Il représente la structure HTML sous forme d'un arbre où chaque élément est un nœud que l'on peut modifier dynamiquement.

Par exemple :

```
<!DOCTYPE html>
<html>
  <head>
    <title>Test page</title>
    <style>
      h1 { color: red; }
    </style>
  </head>
  <body>
    <h1>My Page</h1>
    <p>Hello World!</p>
    
  </body>
</html>
```



1. Sélectionner un élément HTML par DOM

1.1 getElementById()

Sélectionne un élément HTML par son **id**.

Syntaxe :

```
document.getElementById("idElement");
```

Exemple :

```
<p id="titre">Bonjour !</p>
<script>
  var element = document.getElementById("titre");
  console.log(element.innerHTML); // Affiche : Bonjour !
</script>
```

1.2 getElementsByTagName()

Sélectionne tous les éléments d'un certain type (balise HTML).

Syntaxe :

```
document.getElementsByTagName("nomDeBalise");
```

Exemple :

```
<p>Premier paragraphe</p>
<p>Deuxième paragraphe</p>

<script>
  var paragraphes = document.getElementsByTagName("p");
  console.log(paragraphes[0].innerHTML); // Affiche : Premier paragraphe
</script>
```

1.3 getElementsByClassName()

Sélectionne tous les éléments ayant une classe spécifique.

Syntaxe :

```
document.getElementsByClassName("nomDeClasse");
```

Exemple

```
<p class="texte">Bonjour !</p>
<p class="texte">Salut !</p>

<script>
  var elements = document.getElementsByClassName("texte");
  console.log(elements[1].innerHTML); // Affiche : Salut !
</script>
```

1.4 getElementByName()

Sélectionne tous les éléments ayant un nom spécifique d'un attribut.

Syntaxe :

```
document.getElementByName("nomElement");
```

Exemple :

```
<input type="radio" name="genre" value="Homme"> Homme
<input type="radio" name="genre" value="Femme"> Femme

<script>
  var elements = document.getElementByName("genre");
  console.log(elements.length); // Affiche : 2
</script>
```

1.5 querySelector()

Sélectionne le premier élément correspondant à un sélecteur CSS.

Syntaxe :

```
document.querySelector("sélecteurCSS");
```

Exemple avec une classe (.) :


```

<p class="texte">Premier paragraphe</p>
<p class="texte">Deuxième paragraphe</p>

<script>
  var element = document.querySelector(".texte");
  console.log(element.innerHTML); // Affiche : Premier paragraphe
</script>

```

Exemple avec un id (#) :

```

<p id="paragraphe">Contenu</p>

<script>
  var element = document.querySelector("#paragraphe");
  console.log(element.innerHTML); // Affiche : Contenu
</script>

```

Exemple avec une balise HTML :

```

<p>Texte 1</p>
<p>Texte 2</p>

<script>
  var element = document.querySelector("p");
  console.log(element.innerHTML); // Affiche : Texte 1
</script>

```

1.6 querySelectorAll()

Sélectionne tous les éléments correspondant à un sélecteur CSS.

Syntaxe :

```
document.querySelectorAll("sélecteurCSS");
```

Exemple :

```
<p class="texte">Bonjour !</p>
<p class="texte">Salut !</p>

<script>
  var elements = document.querySelectorAll(".texte");
  console.log(elements[1].innerHTML); // Affiche : Salut !
</script>
```

2. Modifier des éléments HTML

2.1 Modifier le texte d'un élément (innerHTML ou textContent)

```
document.getElementById("monID").innerHTML = "Nouveau texte";
```

Exemple :

```
<p id="titre">Ancien texte</p>

<script>
  document.getElementById("titre").innerHTML = "Nouveau texte";
</script>
```

```
<div id="exemple">Ancien texte</div>

<script>
  document.getElementById("exemple").textContent = "<i>Nouveau texte</i>";
</script>
```

2.2 getAttribute() - Récupérer un attribut

```
var valeur = document.getElementById("monElement").getAttribute("nomAttribut");
```

Exemple :

```


<script>
  var source = document.getElementById("image").getAttribute("src");
  console.log(source); // Affiche : image.jpg
</script>
```

2.3 setAttribute() - Modifier un attribut

```
document.getElementById("monElement").setAttribute("nomAttribut", "valeur");
```

Exemple :

```


<script>
  document.getElementById("image").setAttribute("src", "nouvelle_image.jpg");
</script>
```

2.4 appendChild() - Ajouter un élément enfant

```
document.getElementById("parent").appendChild(nouvelElement);
```

Exemple :

```
<div id="conteneur"></div>

<script>
  var nouveauParagraphe = document.createElement("p");
  nouveauParagraphe.innerHTML = "Texte ajouté dynamiquement";
  document.getElementById("conteneur").appendChild(nouveauParagraphe);
</script>
```

2.5 removeChild() - Supprimer un élément enfant

```
document.getElementById("parent").removeChild(enfant);
```

Exemple :

```

<div id="conteneur">
  <p id="texte">Texte à supprimer</p>
</div>

<script>
  var parent = document.getElementById("conteneur");
  var enfant = document.getElementById("texte");
  parent.removeChild(enfant);
</script>

```

3. Modifier un Style en Accédant Directement à style

La manière la plus simple de modifier le style d'un élément est d'utiliser `element.style.propriétéCSS = "valeur";`.

Exemple : Changer la couleur et la taille d'un titre

```

// Sélection de l'élément par son ID
const titre = document.getElementById("monTitre");

// Modification du style
titre.style.color = "blue"; // Change la couleur du texte en bleu
titre.style.fontSize = "24px"; // Change la taille du texte
titre.style.fontWeight = "bold"; // Met le texte en gras

```

♦ **Remarque :** Les noms des propriétés CSS en plusieurs mots (`background-color`, `font-size`, etc.) doivent être écrits en `camelCase` (`backgroundColor`, `fontSize`).

4. Les événements en JavaScript

3.1 `addEventListener()` avec click

Ajoute un événement qui s'exécute lorsqu'on clique sur un élément.

Syntaxe :

```

element.addEventListener("click", fonction);

```

Exemple :

```
<button id="monBouton">Cliquez-moi</button>

<script>
  document.getElementById("monBouton").addEventListener("click", function() {
    alert("Bouton cliqué !");
  });
</script>
```

3.2 addEventListener() avec dblclick

Ajoute un événement qui s'exécute lors d'un double clic.

Exemple :

```
<button id="monBouton">Double-cliquez-moi</button>

<script>
  document.getElementById("monBouton").addEventListener("dblclick", function() {
    alert("Bouton double-cliqué !");
  });
</script>
```

3.3 addEventListener() avec mouseenter

Ajoute un événement qui s'exécute lorsque la souris entre sur l'élément.

Exemple :

```
<p id="monParagraphe">Passez la souris ici</p>

<script>
  document.getElementById("monParagraphe").addEventListener("mouseenter", function() {
    alert("Souris détectée !");
  });
</script>
```